Sri Eshwar College of Engineering
Coimbatore | Tamilnadu
An Autonomous Institution
Affiliated to Anna University, Chennai

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

U23IT483 BLOCKCHAIN TECHNOLOGIES

SRI ESHWAR COLLEGE OF ENGINEERING

KINATHUKADAVU

COIMBATORE - 641202

Sri Eshwar College of Engineering
Coimbatore | Tamilnadu
An Autonomous Institution
Affiliated to Anna University, Chennai

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

BONAFIDE CERTIFICATE

Certified that this is the bonafide record of work done by Mr./Ms. .....
Register No: ..... of ..... Year B.E./B.Tech. - ..... in the **U23IT483 BLOCKCHAIN TECHNOLOGIES** laboratory during the ..... Semester of the academic year **2026 - 2027.**

Signature of faculty In-charge

Head of the Department

Submitted for the practical examination held on .....

Internal Examiner

External Examiner

LIST OF EXPERIMENTS

EXP. NO	DATE	NAME OF THE EXPERIMENT	CO	PAGE NO.	MARKS	SIGN.
1		Merkle Tree-based Verification System for Decentralized Storage	CO1	1		
2		Custom Blockchain Architecture	CO2	6		
3		Functional Blockchain Prototype	CO3	15		
4		Blockchain Prototype with Consensus	CO4	19		
5		Decentralized Cryptocurrency Wallet	CO5	24		
6		Smart Contract-Based Voting System	CO5	27		

Average Marks: Signature of the Faculty

Ex. No. 1	Merkle Tree-based Verification System for Decentralized Storage

Aim

Design and implement an optimized Merkle Tree-based verification system to secure large-scale decentralized data storage. This experiment demonstrates data integrity verification, efficient retrieval proofs, and basic fraud-prevention checks using cryptographic hash functions (SHA-256).

Procedure

1. Create a dedicated project directory MerkleTreeLab and within it, add a subfolder named sample_files. Populate this folder with 4–8 small text files (e.g., file1.txt, file2.txt, etc.) to serve as data blocks for verification.
2. Develop helper functions in Python to read each file, compute its SHA-256 hash, and convert the output into a hexadecimal format. These will serve as the *leaf hashes* in the Merkle Tree.
3. Use the computed leaf hashes as input to build the Merkle Tree. Iteratively hash each consecutive pair of child nodes until a single *root hash* is obtained, representing the integrity of the entire dataset.
4. Save all leaf hashes and the final root hash into a metadata file (or in-memory structure). This simulates how decentralized storage systems or blockchains store verification references securely on-chain or in a trusted ledger.
5. Select any leaf node (file) and compute its *Merkle proof*, which consists of the sequence of sibling hashes along the path from that leaf

to the root. This proof enables verification without reprocessing the entire dataset.

6. Recompute the root hash from the selected leaf and its proof. Compare the computed root with the stored root hash. If they match, the file is verified as authentic and untampered.
7. Modify one of the sample text files and re-run the verification process. Observe that the recomputed root hash differs from the stored root, confirming successful tampering detection and proving the effectiveness of the Merkle verification mechanism.

Code

```
# merkle_demo.py - Merkle Tree (SHA-256)
import os, hashlib, sys


def sha256_hex(data: bytes) -> str:
    return hashlib.sha256(data).hexdigest()

def file_hash(path: str) -> str:
    with open(path, "rb") as f:
        return sha256_hex(f.read())


def merkle_layer(hashes):
    # If odd, duplicate last
    if len(hashes) % 2 == 1:
        hashes = hashes + [hashes[-1]]
    parent = []
    for i in range(0, len(hashes), 2):
        combined = bytes.fromhex(hashes[i]) + bytes.fromhex(hashes[i+1])
        parent.append(sha256_hex(combined))
    return parent
```

```
def build_merkle_tree(file_paths):
    leaves = [file_hash(p) for p in file_paths]
    tree = [leaves]
    layer = leaves
    while len(layer) > 1:
        layer = merkle_layer(layer)
        tree.append(layer)
    return tree  # bottom-up layers, tree[-1][0] is root

def merkle_proof(tree, index):
    proof = []
    idx = index
    for layer in tree[:-1]:
        # sibling index
        if idx % 2 == 0:
            sib = idx + 1 if idx + 1 < len(layer) else idx
        else:
            sib = idx - 1
        proof.append(layer[sib])
        idx = idx // 2
    return proof

def verify_proof(leaf_hash, proof, root, index):
    computed = leaf_hash
    idx = index
    for sib in proof:
        if idx % 2 == 0:
            combined = bytes.fromhex(computed) + bytes.fromhex(sib)
        else:
            combined = bytes.fromhex(sib) + bytes.fromhex(computed)
        computed = sha256_hex(combined)
        idx = idx // 2
    return computed == root
```

```
if __name__ == "__main__":
    # Example usage:
    folder = "sample_files"
    files = sorted([os.path.join(folder, f) for f in os.listdir(folder) if
f.endswith(".txt")])
    tree = build_merkle_tree(files)
    root = tree[-1][0]
    print("Files and leaf hashes:")
    for f, h in zip(files, tree[0]):
        print(f"{f} -> {h}")
    print("\\nRoot hash:", root)
    # Proof for second file (index 1)
    proof = merkle_proof(tree, 1)
    print("\\nProof for index 1:", proof)
    ok = verify_proof(tree[0][1], proof, root, 1)
    print("\\nVerification result:", ok)
```

Output

```
$ python3 merkle_demo.py
Leaf file 'file1.txt'  -> H1 = 3a7bd3...f0
Leaf file 'file2.txt'  -> H2 = b4c2e1...9d
Leaf file 'file3.txt'  -> H3 = 7f1a2c...55
Leaf file 'file4.txt'  -> H4 = a9c0f4...12

Internal H12 = hash(H1||H2) -> c8e4f1...ab
Internal H34 = hash(H3||H4) -> 5b3d2a...ee

Root hash = hash(H12||H34) -> d4f9a0...77

Verification: file3.txt included? True
Recomputed root matches stored root -> Verification SUCCESS
```

```
SHA-256 hashing steps (example)
1. Read file bytes
2. H1 = sha256(file1)
3. H2 = sha256(file2)
4. H12 = sha256(H1 || H2)
5. Continue up to Root
```

```
$ python3 merkle_demo.py
Root Hash: d4f9a0e8...77
Verification Result: True

$ echo 'tampered data' > sample_files/file2.txt
$ python3 merkle_demo.py
Root Hash: 9a7e2c1b...11
Verification Result: False
```

Result

The program for the Merkle Tree-based Verification Systemfor Decentralized Storage was executed successfully.The system effectively constructed a Merkle Tree using cryptographic SHA-256 hash functions and generated a unique root hash representing the integrity of the entire dataset.

Ex. No. 2	Custom Blockchain Architecture

Aim

Design and implement a custom blockchain architecture by developing a block structure that includes essential components such as block header, previous hash, timestamp, Merkle root, nonce, and cryptographic signature to ensure integrity and authenticity of blocks.

Procedure

1. Create a project folder (e.g., CustomBlockchain). Install the ecdsa library

   pip install ecdsa
2. Create a Block class that contains: index, timestamp, transactions (list), previous_hash, merkle_root, nonce, signature, and public_key (signer).
3. Compute cryptographic values and Implement SHA-256 hashing for block headers.
4. Implement a deterministic compute_merkle_root that computes the Merkle root from the block's transactions (handling odd leaf counts by duplicating the last hash).
5. Generate an ECDSA key pair (RSA/ECDSA).
6. Implement sign_block(sk) to sign the block header hash and store the signature and public key.
7. Implement verify_signature() to validate the signature using the stored public key.

8. Create a Blockchain manager with methods to create_genesis_block(), add_block(transactions, signer_sk), and validate_chain().
9. On add_block, compute the Merkle root, optionally run a simple PoW loop (nonce) if desired, compute header hash, sign block with provided signer key, and append.
10. Implement chain validation to verify previous_hash linkage, recompute Merkle root, recompute header hash, and verify signatures for every block.
11. Create a small demo in if __name__ == "__main__": that generates a keypair, creates genesis, adds a couple of blocks, prints root/hashes/signatures, and runs validate_chain().

Code

```
import time
import json
import hashlib
from typing import List, Optional, Dict, Any
from ecdsa import SigningKey, VerifyingKey, SECP256k1,
BadSignatureError

def sha256_hex(data: bytes) -> str:
    return hashlib.sha256(data).hexdigest()

def compute_merkle_root(transactions: List[str]) -> str:
    if not transactions:
        # Conventional choice: hash of empty string
        return sha256_hex(b"")
```

```
    # Start with leaf hashes
    layer = [sha256_hex(tx.encode()) for tx in transactions]

    while len(layer) > 1:
        if len(layer) % 2 == 1:
            layer.append(layer[-1])
        parent = []
        for i in range(0, len(layer), 2):
            combined = bytes.fromhex(layer[i]) + bytes.fromhex(layer[i +
1])
            parent.append(sha256_hex(combined))
        layer = parent
    return layer[0]

class Block:
    def __init__(
        self,
        index: int,
        transactions: List[str],
        previous_hash: str,
        signer_pubkey_hex: Optional[str] = None,
    ):
        self.index = index
        self.timestamp = time.time()
        self.transactions = transactions.copy()
        self.previous_hash = previous_hash
```

```
        self.merkle_root = compute_merkle_root(self.transactions)
        self.nonce = 0
        self.signature: Optional[str] = None  # hex string
        self.signer_pubkey_hex = signer_pubkey_hex
    def header_dict(self) -> Dict[str, Any]:
        return {
            "index": self.index,
            "timestamp": round(self.timestamp, 6),  # rounded for
deterministic representation
            "previous_hash": self.previous_hash,
            "merkle_root": self.merkle_root,
            "nonce": self.nonce,
        }

    def compute_header_hash(self) -> str:
        header_json = json.dumps(self.header_dict(), sort_keys=True,
separators=(",", ":"))
        return sha256_hex(header_json.encode())

    def sign_block(self, signing_key: SigningKey):
        header_hash = self.compute_header_hash()
        sig_bytes = signing_key.sign(header_hash.encode())
self.signature = sig_bytes.hex()
        self.signer_pubkey_hex =
signing_key.get_verifying_key().to_string("compressed").hex()

    def verify_signature(self) -> bool:
```

```
        if not (self.signature and self.signer_pubkey_hex):
            return False
        try:
            vk_bytes = bytes.fromhex(self.signer_pubkey_hex)
            vk = VerifyingKey.from_string(vk_bytes, curve=SECP256k1,
hashfunc=hashlib.sha256)
            header_hash = self.compute_header_hash()
            return vk.verify(bytes.fromhex(self.signature),
header_hash.encode())
        except (BadSignatureError, Exception):
            return False

    def mine(self, difficulty_prefix: str = "0000", max_attempts: int =
5_000_000):
        attempts = 0
        while attempts < max_attempts:
            h = self.compute_header_hash()
            if h.startswith(difficulty_prefix):
                return True
            self.nonce += 1
            attempts += 1
        return False

class Blockchain:
    def __init__(self, difficulty_prefix: str = "0000"):
        self.chain: List[Block] = []
        self.difficulty_prefix = difficulty_prefix
```

```
        self.create_genesis_block()

    def create_genesis_block(self):
        genesis = Block(index=0, transactions=["Genesis Block"],
previous_hash="0" * 64)
        self.chain.append(genesis)

    def last_hash(self) -> str:
        return self.chain[-1].compute_header_hash()

    def add_block(self, transactions: List[str], signing_key:
SigningKey, do_mine: bool = True) -> Block:
        new_block = Block(index=len(self.chain),
transactions=transactions, previous_hash=self.last_hash())
        # optional mining to satisfy difficulty
        if do_mine and self.difficulty_prefix:
            mined = new_block.mine(self.difficulty_prefix)
            if not mined:
        new_block.sign_block(signing_key)
        self.chain.append(new_block)
        return new_block

    def validate_chain(self) -> bool:
            for i in range(1, len(self.chain)):
            prev_block = self.chain[i - 1]
            block = self.chain[i]
            # 1) check previous hash linkage
```

```
            expected_prev_hash = prev_block.compute_header_hash()
            if block.previous_hash != expected_prev_hash:
                print(f"Invalid previous_hash at block {block.index}")
                return False
            # 2) check merkle root
            recomputed_merkle =
compute_merkle_root(block.transactions)
            if recomputed_merkle != block.merkle_root:
                print(f"Merkle root mismatch at block {block.index}")
                return False
            # 3) check signature
            if not block.verify_signature():
                print(f"Signature verification failed at block {block.index}")
                return False
        # verify genesis signature if present
        if len(self.chain) > 0:
            if self.chain[0].signature and not
self.chain[0].verify_signature():
                print("Genesis signature invalid")
                return False
        return True

if __name__ == "__main__":
    # 1) generate ECDSA keypair (signer)
    sk = SigningKey.generate(curve=SECP256k1)  # private key
    vk = sk.get_verifying_key()  # public key
    print("Signer public key (compressed hex):",
```

```
        vk.to_string("compressed").hex()[:64], "...")
    bc = Blockchain(difficulty_prefix="000")
    bc.chain[0].sign_block(sk)
    print("Genesis header hash:", bc.chain[0].compute_header_hash())
    print("Genesis signature (truncated):", (bc.chain[0].signature or
        "")[:64], "...")
    txs_block1 = ["Alice pays Bob 10", "Bob pays Carol 3"]
    b1 = bc.add_block(txs_block1, signing_key=sk, do_mine=True)
    print(f"Added Block {b1.index} | Header Hash:
        {b1.compute_header_hash()} | Nonce: {b1.nonce}")
    print("Block 1 Merkle Root:", b1.merkle_root)
    print("Block 1 signature (truncated):", b1.signature[:64], "...")
    txs_block2 = ["Dave pays Erin 7"]
    b2 = bc.add_block(txs_block2, signing_key=sk, do_mine=True)
    print(f"Added Block {b2.index} | Header Hash:
{b2.compute_header_hash()} | Nonce: {b2.nonce}")
    print("Block 2 Merkle Root:", b2.merkle_root)
    valid = bc.validate_chain()
    print("Chain validation result:", valid)
    print("\n-- Tamper test: modify transaction in Block 1 --")
    bc.chain[1].transactions[0] = "Alice pays Bob 9999"  # tamper
    bc.chain[1].merkle_root = compute_merkle_root(bc.chain[1].
        transactions)
    valid_after_tamper = bc.validate_chain()
    print("Chain validation after tamper (should be False):",
        valid_after_tamper)
```

Output

```
=== Blockchain Data ===
Block 0
Timestamp: 1762531311.8059807
Merkle Root: 89eb0ac031a63d2421cd05a2fbe41f3ea35f5c3712ca839cbf6b85c4ee07b7a3
Nonce: 252
Hash: 000fd21de2c1a9d1d3f8bddb5094810c24e94ed421a61ca6da46541c295e0dc4
Prev Hash: 0000000000000000000 ...

Block 1
Timestamp: 1762531311.8077476
Merkle Root: 9dc65d39f9ecc6f623246e2dd7ac8142e9b14be283ea914385f8d411e76985c5
Nonce: 2304
Hash: 000fd8e8704caa4c483afff77b3ae1754c215c0448b474534bef23dc02505ca4
Prev Hash: 000fd21de2c1a9d1d3f8 ...

Block 2
Timestamp: 1762531311.822814
Merkle Root: e5a26cbb23d874911994866d5c667ac82322e5d592abb0ecd672baba0e37b908
Nonce: 8841
Hash: 0000b0a00a96a645e1dfca62383973d15aec9e5a93761f6065ce3fb0fae15b19
Prev Hash: 000fd8e8704caa4c483a ...

Blockchain valid: True

--- Tamper Test ---
Blockchain valid after tamper: False
```

Result

Created a secure blockchain structure where each block is linked by cryptographic hashes, validated through Merkle roots, and signed using ECDSA digital signatures.The verification process confirmed the integrity of all blocks, and tampering detection worked correctly by invalidating the chain after modification.

Ex. No. 3	Functional Blockchain Prototype

Aim

Design and develop a fully functional blockchain prototype that includes secure block creation, transaction validation, and a consensus mechanism to ensure integrity and immutability of transactions across the network.

Procedure

1. Create a new Python project folder named BlockchainPrototype.
2. Implement two classes — Block and Blockchain — to manage block creation, validation, and linkage.
3. Add a transaction pool where valid transactions are collected before block creation.
4. Implement a Proof-of-Work mechanism by finding a nonce value that produces a hash beginning with predefined zeros (e.g., “0000”).
5. Create a function to verify block integrity by comparing stored and recomputed hashes.
6. Simulate the mining of new blocks by adding validated transactions and linking them using the previous hash.
7. Print the blockchain to confirm proper linkage and proof-of-work execution.

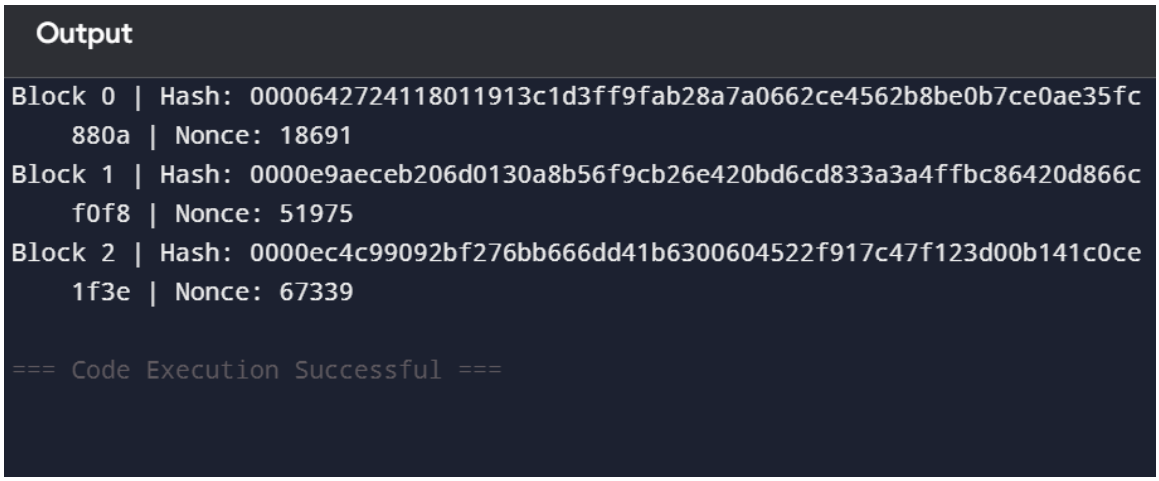
Code

```
import hashlib, json, time
class Block:
    def __init__(self, index, transactions, previous_hash):
        self.index = index
        self.timestamp = time.time()
        self.transactions = transactions
        self.previous_hash = previous_hash
        self.nonce, self.hash = self.mine_block()
    def compute_hash(self, nonce):
        block_string = json.dumps({
            "index": self.index,
            "timestamp": self.timestamp,
            "transactions": self.transactions,
            "previous_hash": self.previous_hash,
            "nonce": nonce
        }, sort_keys=True)
        return hashlib.sha256(block_string.encode()).hexdigest()
    def mine_block(self):
        nonce = 0
        while True:
            hash_val = self.compute_hash(nonce)
            if hash_val.startswith("0000"):
                return nonce, hash_val
```

```
            nonce += 1
        class Blockchain:
            def __init__(self):
                self.chain = []
                self.pending_transactions = []
                self.create_genesis_block()
            def create_genesis_block(self):
                genesis_block = Block(0, ["Genesis Block"], "0")
                self.chain.append(genesis_block)
            def add_transaction(self, sender, receiver, amount):
                self.pending_transactions.append({
                    "sender": sender,
                    "receiver": receiver,
                    "amount": amount
                })
            def add_block(self):
                new_block = Block(len(self.chain), self.pending_transactions, self.chain[-1].hash)
                self.chain.append(new_block)
                self.pending_transactions = []
        # Example Run
        if __name__ == "__main__":
            bchain = Blockchain()
            bchain.add_transaction("Alice", "Bob", 50)
            bchain.add_block()
```

```
bchain.add_transaction("Charlie", "David", 20)
bchain.add_block()
for blk in bchain.chain:
    print(f"Block {blk.index} | Hash: {blk.hash} | Nonce: {blk.nonce}")
```

Output



Result

The blockchain was constructed with validated transactions, each block linked using the previous block's hash. The proof-of-work algorithm ensured consensus, immutability, and security of the blockchain.

Ex. No. 4	Blockchain Prototype with Consensus

Aim

Design and develop a blockchain prototype that includes the core functionalities of block creation, transaction validation, cryptographic security, and a decentralized consensus mechanism ensuring all nodes agree on the same valid chain.

Procedure

1. Create a folder named ConsensusBlockchain and a Python file consensus_blockchain.py.
2. Implement a Block class containing the following: index, timestamp, transactions, previous_hash, nonce, and hash.
3. Implement a method *mine_block()* performing simple proof-of-work.
4. Create the Blockchain class and maintain a list of blocks.
5. Initializes the chain *create_genesis_block()*
6. adds new mined blocks *add_block(transactions)*
7. validates hashes and linkage *is_valid()*
8. To simulate multiple nodes, instantiate three or more blockchain objects (node1, node2, node3).
9. Allow each node to add slightly different transactions to its local chain.
10. To implement consensus mechanism, create a function consensus(chains) that compares the chain lengths of all nodes and replaces shorter or invalid chains with the longest valid one (longest-chain rule).

11. Print each node's chain length before and after consensus.Confirm that all nodes hold the same chain after synchronization.

Code

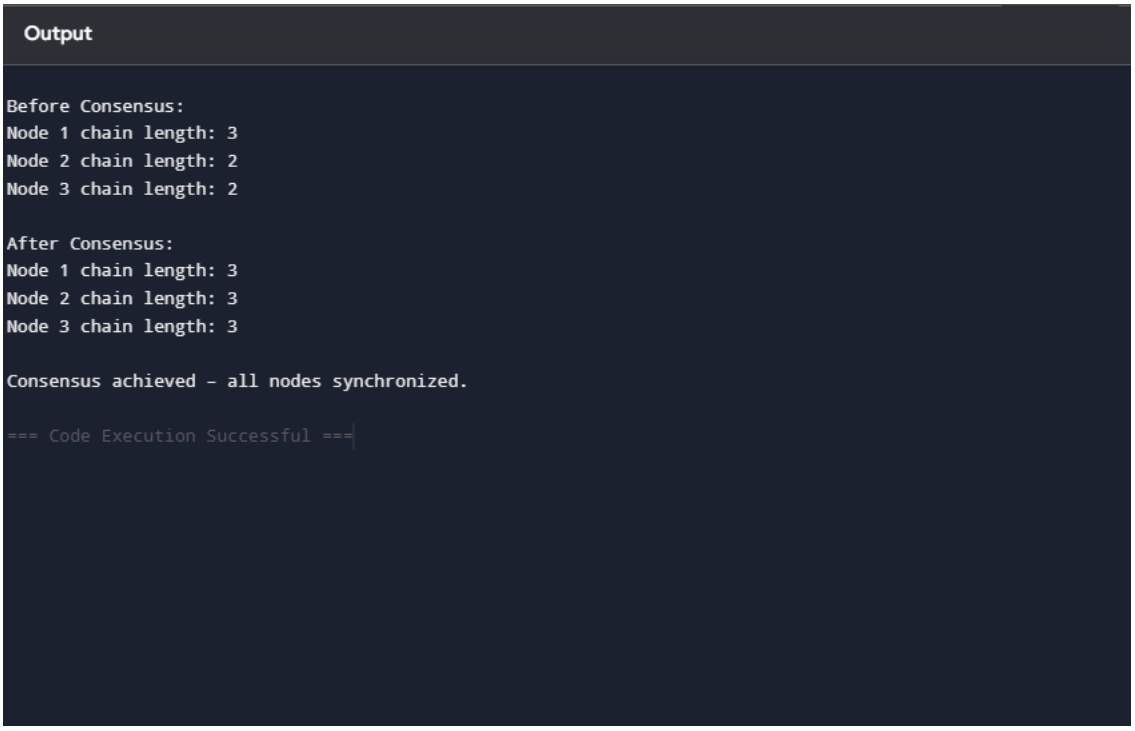
```
import hashlib, json, time, random
class Block:
    def __init__(self, index, transactions, previous_hash):
        self.index = index
        self.timestamp = time.time()
        self.transactions = transactions
        self.previous_hash = previous_hash
        self.nonce, self.hash = self.mine_block()
    def compute_hash(self, nonce):
        block_string = json.dumps({
            "index": self.index,
            "timestamp": self.timestamp,
            "transactions": self.transactions,
            "previous_hash": self.previous_hash,
            "nonce": nonce
        }, sort_keys=True)
        return hashlib.sha256(block_string.encode()).hexdigest()
    def mine_block(self, difficulty="000"):
        nonce = 0
        while True:
```

```
            hash_val = self.compute_hash(nonce)
            if hash_val.startswith(difficulty):
                return nonce, hash_val
            nonce += 1
# ---------- Blockchain Definition ----------
class Blockchain:
    def __init__(self):
        self.chain = []
        self.create_genesis_block()
    def create_genesis_block(self):
        genesis = Block(0, ["Genesis Block"], "0" * 64)
        self.chain.append(genesis)
    def add_block(self, transactions):
        prev_hash = self.chain[-1].hash
        block = Block(len(self.chain), transactions, prev_hash)
        self.chain.append(block)
    def is_valid(self):
        for i in range(1, len(self.chain)):
            prev, curr = self.chain[i-1], self.chain[i]
            if curr.previous_hash != prev.hash:
                return False
            if curr.hash != curr.compute_hash(curr.nonce):
                return False
```

```
        return True
# ---------- Consensus Function ----------
def consensus(chains):
    longest_chain = None
    max_len = 0
    for chain in chains:
        if chain.is_valid() and len(chain.chain) > max_len:
            longest_chain = chain.chain
            max_len = len(chain.chain)
    # Replace all other chains
    for bc in chains:
        bc.chain = list(longest_chain)
    return longest_chain
# ---------- Demonstration ----------
if __name__ == "__main__":
    # Simulate three nodes
    node1, node2, node3 = Blockchain(), Blockchain(), Blockchain()
    # Each node mines different transactions
    node1.add_block(["A pays B 5"])
    node1.add_block(["C pays D 7"])
    node2.add_block(["X pays Y 10"])
    node3.add_block(["M pays N 2", "N pays O 4"])
    print("\nBefore Consensus:")
```

```
for i, n in enumerate([node1, node2, node3], start=1):
    print(f"Node {i} chain length: {len(n.chain)}")
# Run consensus algorithm
consensus([node1, node2, node3])
print("\nAfter Consensus:")
for i, n in enumerate([node1, node2, node3], start=1):
    print(f"Node {i} chain length: {len(n.chain)}")
print("\nConsensus achieved – all nodes synchronized.")
```

Output



Result

The program for the Blockchain Prototype with Consensus executed successfully.Multiple nodes initially maintained different chains.After running the consensus algorithm, all nodes adopted the longest valid chain, ensuring data consistency and decentralized agreement.

Ex. No.5	Decentralized Cryptocurrency Wallet

Aim

Design and develop a secure, decentralized cryptocurrency wallet that allows users to generate keys, store addresses, send and receive transactions across blockchain networks in a simplified test environment.

Procedure

1. Create a new folder named CryptoWallet and a file wallet.py.
2. To generate wallet keys, use Python's built-in secrets and hashlib to generate a secure random private key (hexadecimal).
3. Derive a public address by hashing the private key using SHA-256.
4. To simulate wallet functions, implement *create_wallet()* to generate and store keys.
5. Implement *get_balance(address)* that simulates querying a mock ledger dictionary.
6. Implement *send_funds(sender, receiver, amount)* to deduct and credit balances in this mock ledger.
7. To maintain local ledger, use a Python dictionary as a simple in-memory ledger for demonstration.
8. Update balances after each simulated transaction.
9. Display wallet information
10. Print private key, public address, balances, and transaction confirmation.
11. Verify that balances update correctly and that only the wallet owner (with the private key) can initiate transactions.

Code

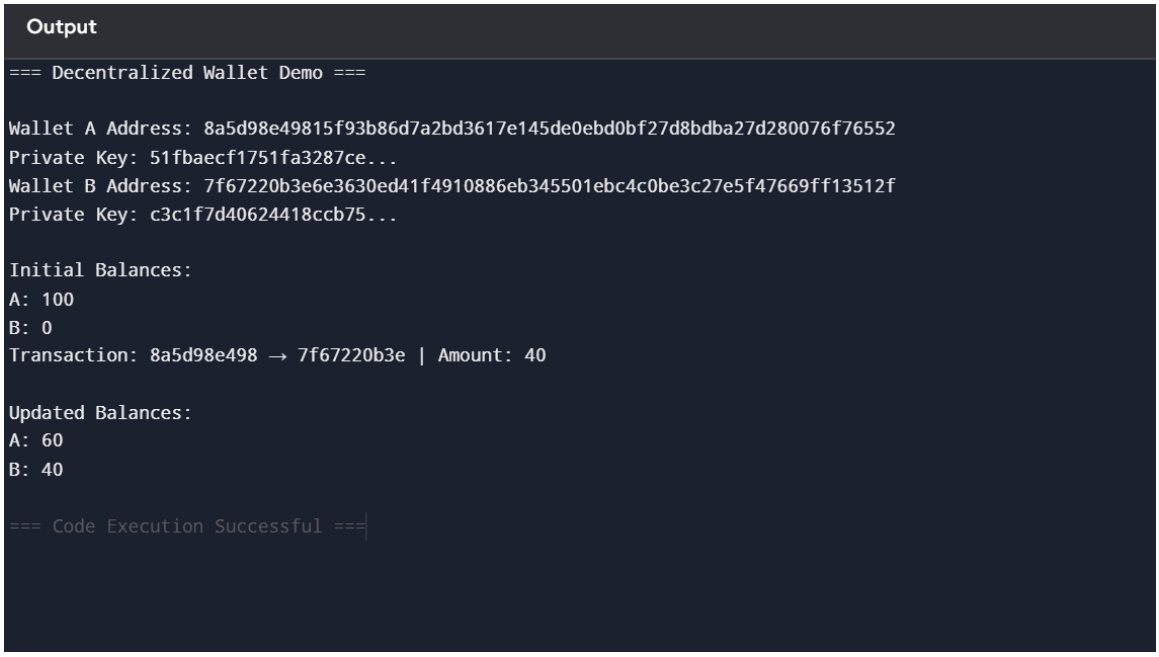
```
import secrets, hashlib

def create_wallet():
```

```
    private_key = secrets.token_hex(32)
    public_key = hashlib.sha256(private_key.encode()).hexdigest()
    return private_key, public_key
ledger = {}
def get_balance(address):
    return ledger.get(address, 0)
def send_funds(sender, receiver, amount):
    if ledger.get(sender, 0) < amount:
        print("Insufficient balance.")
        return False
    ledger[sender] -= amount
    ledger[receiver] = ledger.get(receiver, 0) + amount
    print(f"Transaction: {sender[:10]} → {receiver[:10]} | Amount: {amount}")
    return True
if __name__ == "__main__":
    print("=== Decentralized Wallet Demo ===")
    privA, addrA = create_wallet()
    privB, addrB = create_wallet()
    ledger[addrA] = 100  # initial balance for demo
    print(f"\nWallet A Address: {addrA}\nPrivate Key: {privA[:20]}...")
    print(f"Wallet B Address: {addrB}\nPrivate Key: {privB[:20]}...")
    print("\nInitial Balances:")
```

```
print("A:", get_balance(addrA))
print("B:", get_balance(addrB))
send_funds(addrA, addrB, 40)
print("\nUpdated Balances:")
print("A:", get_balance(addrA))
print("B:", get_balance(addrB))
```

Output



Result

The program for the Decentralized Cryptocurrency Wallet executed successfully.

Ex. No. 6	Smart Contract-Based Voting System

Aim

Design and implement a blockchain-based voting system using Python simulation or Solidity smart contracts that ensures transparency, immutability, and fairness in decentralized elections.

Procedure

1. Create a folder named VotingSystem and file voting_blockchain.py.
2. Each block contains voter_id, candidate, and timestamp.
3. Previous hash ensures that votes are tamper-proof.
4. Add validation by implement voter verification to ensure no duplicate voting.
5. Compute SHA-256 hash for each vote block.
6. Simulate voting process that accept votes from different users.
7. After all votes are casted, generate a blockchain where each vote is a block.
8. Traverse all blocks and count candidate votes.
9. Display winner and verify the integrity of the entire chain.

Code

```
import hashlib, json, time

class VoteBlock:

   def __init__(self, voter_id, candidate, previous_hash):

      self.voter_id = voter_id

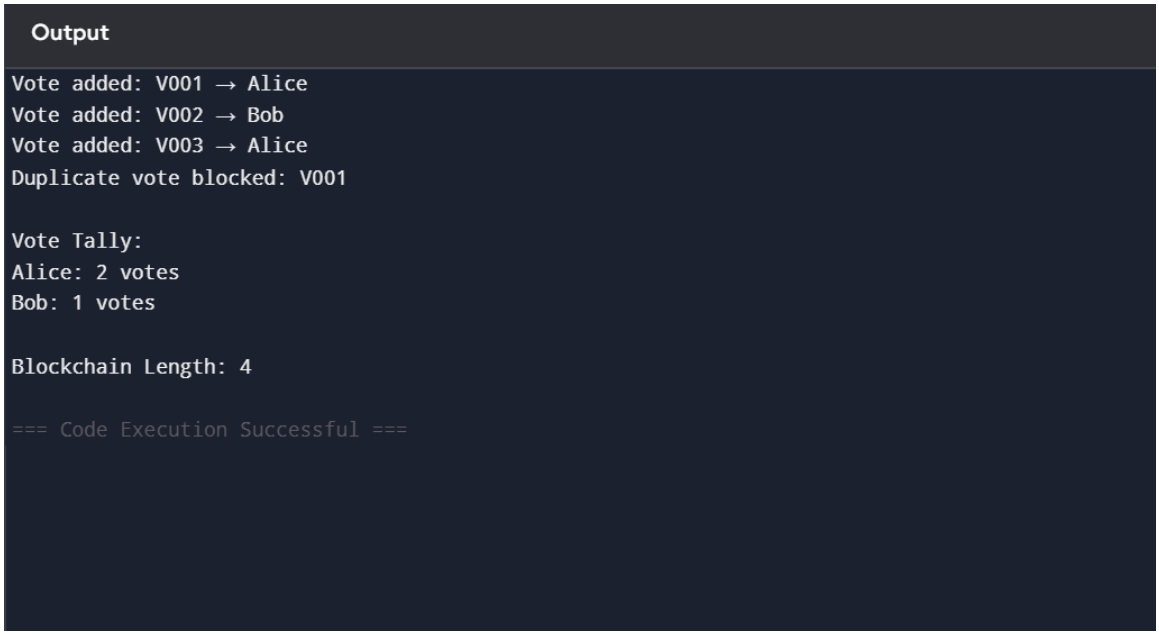
      self.candidate = candidate

      self.timestamp = time.time()
```

```
        self.previous_hash = previous_hash
        self.hash = self.compute_hash()
    def compute_hash(self):
        block_str = json.dumps(self.__dict__, sort_keys=True)
        return hashlib.sha256(block_str.encode()).hexdigest()
class VotingBlockchain:
    def __init__(self):
        self.chain = []
        self.voters = set()
        self.create_genesis_block()
    def create_genesis_block(self):
        genesis = VoteBlock("0", "Genesis", "0" * 64)
        self.chain.append(genesis)
    def add_vote(self, voter_id, candidate):
        if voter_id in self.voters:
            print(f"Duplicate vote blocked: {voter_id}")
            return
        prev_hash = self.chain[-1].hash
        new_vote = VoteBlock(voter_id, candidate, prev_hash)
        self.chain.append(new_vote)
        self.voters.add(voter_id)
        print(f"Vote added: {voter_id} → {candidate}")
```

```
    def tally_votes(self):
        results = {}
        for block in self.chain[1:]:
            results[block.candidate] = results.get(block.candidate, 0) + 1
        return results
if __name__ == "__main__":
    vb = VotingBlockchain()
    vb.add_vote("V001", "Alice")
    vb.add_vote("V002", "Bob")
    vb.add_vote("V003", "Alice")
    vb.add_vote("V001", "Alice")  # Duplicate vote attempt
    print("\nVote Tally:")
    tally = vb.tally_votes()
    for candidate, count in tally.items():
        print(f"{candidate}: {count} votes")
    print("\nBlockchain Length:", len(vb.chain))
```

Output



Result

The blockchain-based ledger stored each vote immutably, prevented duplicate voting, and provided transparent vote tallying. This validates the use of blockchain for secure and tamper-proof election systems.